

Failure (a crashing tool, and a bad description) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `tool_selection_practice.py` (Failure section): a crashing tool and a vague description are both real production bugs you will cause yourself at least once — better to see them here, on purpose, where nothing's at stake. **If not:** the Build Task's "one tool designed to sometimes fail" would be the first time you ever turned a raised exception into a clean, model-readable error string.

Basic Version

Approach 1 — both halves, the direct way

```
# tool_selection_practice.py – Failure section
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

# --- Half 1: a crashing tool, made safe ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"

model_with_tools = ChatOpenAI().bind_tools([flaky_divide])
response = model_with_tools.invoke("Use the tool to divide 10 by 0")

for call in response.tool_calls:
    result = flaky_divide.invoke(call["args"])
    print("Tool result:", result) # "Error: division by zero", not a crash

# --- Half 2: a vague description, sharpened ---
```

```

@tool
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"

@tool
def precise_weather(city: str) -> str:
    """Get the current weather (temperature and conditions) for a named city."""
    return f"Sunny in {city}"

def count_calls(weather_tool, runs: int = 5) -> int:
    model_with_tools = ChatOpenAI().bind_tools([weather_tool])
    called = 0
    for _ in range(runs):
        response = model_with_tools.invoke("what's the weather like")
        if response.tool_calls:
            called += 1
    return called

vague_hits = count_calls(vague_weather)
precise_hits = count_calls(precise_weather)
print(f"Vague description: called {vague_hits}/5 times")
print(f"Precise description: called {precise_hits}/5 times")

```

Expected output (example):

```

Tool result: Error: division by zero
Vague description: called 1/5 times
Precise description: called 5/5 times

```

This shows both halves working: an error string instead of a crash, and a measurable difference between a vague and a precise description. It reruns the whole trial from scratch for each half rather than reusing the ambiguity exercise's trial function — fine here, but real code should share that logic instead of duplicating it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the model's reaction to the error, and a swappable-description factory

```
# tool_selection_practice.py – Failure section
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

# --- Half 1: a crashing tool, made safe ---

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        result = a / b
        return str(result)
    except ZeroDivisionError as e:
        return f"Error: {e}"

def demonstrate_safe_failure() -> None:
    # why: proves the caught error string actually reaches the model, and
    # that the model can react sensibly to it instead of treating it as data.
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([flaky_divide])
    response = model_with_tools.invoke("Use the tool to divide 10 by 0")

    for call in response.tool_calls:
        result = flaky_divide.invoke(call["args"])
        print(f"Tool call {call['args']} -> {result}")

    follow_up_prompt = (
        "Use the tool to divide 10 by 0, then tell me in one sentence "
        "what happened."
    )
    follow_up = model_with_tools.invoke(follow_up_prompt)
    reaction = follow_up.content
    print(f"Model's reaction to the error: {reaction}")
```

```
# --- Half 2: a vague description, sharpened ---

def make_weather_tool(description: str):
    # why: builds "before" and "after" tools from the exact same function
    # body, so the description is the only thing that changes between trials.
    def get_weather(city: str) -> str:
        return f"Sunny in {city}"
    get_weather.__doc__ = description
    return tool(get_weather)

def count_tool_calls(weather_tool, prompt: str, runs: int = 5) -> int:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([weather_tool])
    called = 0
    for _ in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called += 1
    return called

def demonstrate_description_fix() -> None:
    prompt = "what's the weather like"

    vague_tool = make_weather_tool("gets data")
    precise_tool = make_weather_tool(
        "Get the current weather (temperature and conditions) for a named city."
    )

    vague_hits = count_tool_calls(vague_tool, prompt)
    precise_hits = count_tool_calls(precise_tool, prompt)

    print(f"Vague description: called {vague_hits}/5 times")
    print(f"Precise description: called {precise_hits}/5 times")

if __name__ == "__main__":
    demonstrate_safe_failure()
    print()
    demonstrate_description_fix()
```

Expected output (example):

```
Tool call {'a': 10, 'b': 0} -> Error: division by zero
Model's reaction to the error: The division failed – you can't divide by zero.
```

Vague description: called 1/5 times
Precise description: called 5/5 times

Difference from Basic: `make_weather_tool()` builds both the "before" and "after" tools from the exact same function body, so the description is the *only* thing that changes between the two trials — this is what makes the comparison actually mean something, instead of accidentally comparing two tools that differ in more than one way. `demonstrate_safe_failure()` also shows the model's *reaction* to the error, not just the raw error string, which is the part that actually matters in production.

Which one should you actually write? Intermediate's version covers what this exercise asks for — a caught error instead of a crash, and a measured before/after description comparison. Two things are worth naming as gaps rather than fixing here: a bare `except Exception` treats a real bug in the tool's own code the same as an expected failure like a divide-by-zero, and a 5-run split is a quick sanity check, not something to act on without a bigger sample. Both are worth reaching for on a tool you're about to keep and reuse for real, not for this exercise.